

QP Frameworks and QM Modeling Tool

In this article, we explore quantum platform (QP) frameworks and modelling tool. The framework can be used to produce highly-responsive and modular-embedded system applications. Design and implementation of real-time embedded software is so easy with the help of the QP modelling- and code-generation tool

JAI SACHITH PAUL

QP, from Quantum Leaps, is a family of open source frameworks that helps a user build efficient real-time embedded applications. The framework can completely replace the need for a traditional real-time operating system (RTOS) on a bare-metal single-chip microcontroller that controls the entire embedded system. The QP family consists of three frameworks: QP/C and QP-nano (written in C) and QP/C++ (written in C++).

Why quantum platform

The QP reveals itself before us as a better alternative to the RTOS in a single-chip microcontroller, controlling the embedded system application. Let us try to figure out the reasons for this.

No blocking. In an RTOS, we have counters, flags and other blocking mechanisms that are time-consuming in the sense that, when a program is blocked, it is not doing any work while waiting for an event. This also consumes a lot of memory space. In order to keep things simple, active objects are introduced into the QP framework by behavioural modelling specified by hierarchical state machine (UML state chart). The QP framework provides the active objects with the execution thread, queuing of events and event based timing services.

The elimination of blocks helps the program to have a smaller footprint in RAM (data space) as well as in ROM (code space). The scalability and ef-

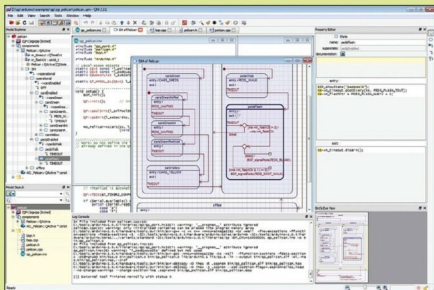


Fig. 1: QM graphical modelling tool



Fig. 2: Components of quantum platform

iciency is thereby increased considerably in a resource-constrained embedded system.

High-efficiency and better resource utilisation. The flow of program is controlled by asynchronous events initiated by user actions, such as a mouse click, sensor outputs or messages from other threads. This event-driven approach helps the programmer to express his event more directly, which, in turn, improves the efficiency of his work. An-

Components in QP architecture and functions

Quantum event processor (QEP). Allows direct coding of UML state charts in C/C++. Supports hierarchical state nesting.

Quantum framework (QF). Enhances concurrent execution of state machines specifically designed for real-time embedded applications.

Quantum kernel (QS). Executes state machines in a run-to-completion fashion.

Quantum spy (QS). Enables live monitoring of QP applications with minimal footprint and slowing down of code.

other advantage of using this scheme is that the CPU needs to be busy at the occurrence of the event only. This means, that, we have a better resource-utilisation and power-efficiency. No wonder the event-driven, quantum platform is